

MoneyPAL NEST Aggregate Blocks (NAB)

Technical Developer Brief

Version 1.0

1. Purpose

The NEST Aggregate Block (NAB) is the canonical business record for every business entity in the MoneyPAL ecosystem.

Unlike a traditional relational database where rows are continuously updated, a NAB records **every business event as an immutable append-only event**.

The current state of an Enterprise, Person, Loan or Household is always reconstructed by replaying its events.

Think of a NAB as a **Git repository for a business object**.

Nothing is overwritten.

Everything is versioned.

Everything is explainable.

2. Design Principles

Every NAB shall satisfy the following principles:

- Immutable
- Append-only
- Event-driven
- Aggregate-oriented
- Time ordered
- Versioned
- Replayable
- Auditable
- AI-readable

- API accessible
-

3. Reference Architecture

The implementation should draw inspiration from modern Event Sourcing architectures rather than blockchain.

Recommended references:

EventStoreDB

One stream per Aggregate.

Enterprise-123

Event 1

Event 2

Event 3

Event 4

Axon Framework

Aggregate Root

↓

Commands

↓

Events

↓

Event Store

↓

Projections

Kafka

Business Events

↓

Event Topics

↓

Consumers

↓

Materialized Views

Git

Commit

↓

History

↓

Checkout

↓

Replay

A NAB behaves very similarly.

4. Aggregate Types

Initially implement the following Aggregate Roots.

Enterprise

Person

Household

Loan

Consent

Document

Branch

Employee

Future aggregates can be added without changing the framework.

5. NAB Structure

Each NAB consists of:

Aggregate

|

|— Metadata

|— Header

|— Event Stream

|— Attachments

|— Links

└— Projections

Only the Event Stream is immutable.

Everything else is derived.

6. Aggregate Metadata

Example

```
{  
  
  "aggregateId":"ENT-000123",  
  
  "aggregateType":"Enterprise",  
  
  "version":42,  
  
  "createdAt":"2026-07-01T10:15:00Z",  
  
  "status":"ACTIVE"  
}
```

7. Event Model

Every event contains:

```
{  
  
  "eventId": "",  
  
  "aggregateId": "",  
  
  "eventType": "",  
  
  "sequence": 15,  
  
  "timestamp": "",  
  
  "actor": "",  
  
  "sourceSystem": "",  
  
  "payload": {},  
  
  "correlationId": "",  
  
  "causationId": ""  
}
```

No event shall ever be modified.

8. Event Types

Example Enterprise events

EnterpriseCreated

EnterpriseUpdated

GSTVerified

AddressChanged

OwnerAdded

OwnerRemoved

LoanRequested

LoanSanctioned

LoanClosed

RiskScoreCalculated

BusinessVisited

ConsentGranted

ConsentRevoked

DocumentUploaded

InspectionCompleted

Every event becomes part of permanent institutional memory.

9. Commands

Commands request changes.

Example

CreateEnterprise

↓

EnterpriseCreated

ApproveLoan



LoanSanctioned

Commands never modify data.

Commands only generate Events.

10. Event Store

Week 1 recommendation:

Use PostgreSQL.

Tables:

Aggregates

Events

Snapshots

Attachments

Links

Later:

Replace Event Store with EventStoreDB if required.

No framework changes should be necessary.

11. Snapshots

Replaying 10,000 events can become expensive.

Every 100 events:

Create Snapshot.

Events

↓

Replay

↓

Snapshot

↓

Replay remaining events

Snapshots are cache.

Events remain source of truth.

12. Projection Layer

Never query Events directly.

Build projections.

Example

Enterprise Projection

Loan Projection

DNBS Projection

Analytics Projection

AI Projection

Workflow Projection

Each projection listens for events.

Updates itself.

Never updates Events.

13. Relationships

Avoid joins.

Store references.

Example

Loan NAB

Enterpriseld

PersonId

HouseholdId

ConsentId

Relationships are reconstructed using IDs.

14. Versioning

Every event increments Version.

Example

Enterprise

Version 1

Enterprise Created

Version 2

GST Verified

Version 3

Address Updated

Version 4

Loan Requested

Current Version = 4

15. Time Travel

One advantage of Event Sourcing.

Question:

"What did we know on 1 Jan?"

Replay until

2026-01-01

Useful for:

Audits

Disputes

AI

Regulators

16. Genesis Migration

Migration produces Events.

Not SQL Inserts.

Prosper Customer

↓

EnterpriseCreated

Prosper Loan

↓

LoanOpened

Ledger Balance

↓

OpeningBalanceLoaded

Migration therefore creates a valid Event History.

17. AI Integration

Every Event becomes semantic context.

Example

LoanSanctioned



Embedding



Vector Store



LLM

The AI Assistant ("Moneypal Genie") answers questions using:

Events

Projections

Attachments

Policies

with source citations.



18. APIs

Required APIs

POST /commands

GET /aggregate/{id}

GET /events/{aggregate}

GET /projection/{id}

GET /history/{aggregate}

GET /snapshot/{id}

POST /projection/rebuild

19. Suggested Technology Stack

Language

Java

or

C#

or

Python

Database

PostgreSQL

Messaging

Kafka (future)

Storage

Object Storage

Search

OpenSearch

Vector Store

pgvector

or Qdrant

Deployment

Docker

Kubernetes

20. Week 1 Deliverables

The NAB team shall deliver:

- Aggregate Root definitions.
 - Event schema (JSON).
 - Command schema.
 - PostgreSQL Event Store schema.
 - Snapshot specification.
 - Projection framework interface.
 - Sample Enterprise NAB.
 - Sample Loan NAB.
 - Sample Person NAB.
 - OpenAPI specification.
 - Developer documentation.
-

21. Definition of Success

The NAB implementation will be considered successful when it can:

1. Create any Aggregate from a sequence of immutable events.
2. Replay events to reconstruct the current state.
3. Produce snapshots without altering event history.
4. Support append-only event creation with optimistic concurrency control.
5. Build independent projections for analytics, regulatory reporting and AI.
6. Accept migrated data from legacy systems by converting records into canonical events rather than database updates.

7. Provide a stable, extensible event model that can support all future MoneyPAL NEST platforms, including MoneyPAL.mp, MoneyPAL.academy and other domain-specific NEST implementations.

MoneyPAL Genesis Layer

Week 1 Development Plan

Objective

By the end of Week 1, the team should have established the architectural foundation of the Genesis Layer. The goal is **not** to complete all features, but to define the core models, interfaces and data contracts that every subsequent sprint will build upon.

Team Structure

Engineer 1 — NAB Platform Lead

Focus

Design the canonical NEST Aggregate Block (NAB) model.

Tasks

- Define the NAB object model.
- Define Aggregate Root concepts.
- Define immutable Event model.
- Define Block metadata.
- Define Event metadata.
- Define Block versioning.
- Define Block identifiers (UUID strategy).
- Define relationships between:
 - Enterprise
 - Person
 - Household
 - Loan
 - Consent
 - Document
- Design append-only event model.
- Develop JSON schema for NABs.
- Create sample NABs from a Prosper loan.

Deliverables

- NAB Specification
 - JSON Schemas
 - Example NAB repository
 - Event model documentation
-

Engineer 2 — Genesis Mapping Lead

Focus

Design the migration layer.

Tasks

Study:

- Prosper database
- Tally exports
- Existing reports

Design:

Canonical Mapping Layer

Source System

↓

Source Object

↓

Field Mapping

↓

Transformation Rules

↓

Canonical Business Object

↓

NAB Event

Produce mapping specifications for:

- Customers
- Enterprises
- Loans
- Guarantors
- Repayment schedules
- Ledger balances

Deliverables

- Source Mapping Specification
 - Mapping Rule Engine Design
 - Field Mapping Catalogue
 - Transformation Rule Catalogue
-

Engineer 3 — Import Framework Lead

Focus

Design standard import files.

Every legacy system should be able to export CSV files into Genesis.

Design canonical CSV specifications for:

- Enterprise.csv
- Person.csv
- Household.csv
- Loan.csv
- LoanAccount.csv
- RepaymentSchedule.csv
- Security.csv
- Guarantor.csv
- Ledger.csv
- Journal.csv
- Collections.csv
- Branch.csv
- Employee.csv

Each specification should define:

- Field Name
- Description
- Data Type
- Mandatory

- Validation
- Example

Deliverables

- CSV Specification
 - Import Validation Rules
 - Sample Import Files
-

Engineer 4 — Intelligence & Reporting Lead

Focus

Two parallel activities.

Activity A

Design Regulatory Reporting.

Study:

- RBI DNBS returns
- RIM
- NAB model

Design:

NAB

↓

Projection Layer

↓

Regulatory Repository

↓

DNBS Reports

Produce:

Projection specifications for:

- DNBS 4A
- DNBS 4B

Identify every NAB field required.

Activity B

Design Moneypal Genie.

Purpose

Natural language interface over NABs.

Example questions:

How many MSME loans do we have?

Total exposure in Belagavi?

Show enterprises with overdue loans.

Which industries grew fastest?

Which guarantors appear on multiple loans?

Why was Loan 123 sanctioned?

Design:

- RAG architecture
- Semantic indexing
- Prompt templates
- SQL generation strategy
- Explainability
- Citation strategy

Deliver:

High-level architecture.

No production AI implementation required during Week 1.

Deliverables

- DNBS Mapping Specification
- Projection Specification
- Genie Architecture
- Example prompts

Daily Plan

Monday

Architecture workshop.

Agree:

- Canonical model
- Repository structure
- Coding standards
- Naming conventions

Tuesday

Individual design work.

Daily design review.

Wednesday

Prototype implementation.

Repository structure.

JSON schemas.

CSV schemas.

Thursday

Cross-review.

Each engineer reviews another engineer's work.

Resolve integration issues.

Friday

Integration day.

Ensure:

NAB ↔ Mapping ↔ CSV ↔ Reporting ↔ Genie all connect logically.

End-of-Week Deliverables

The team should produce:

Foundation

- Canonical NAB Specification
- Aggregate definitions
- Event model
- JSON schemas

Migration

- Source Mapping Specification
- Transformation Rules
- Mapping Catalogue

Import

- Canonical CSV Specifications
- Sample datasets
- Validation Rules

Regulatory

- DNBS 4A field mapping
- DNBS 4B field mapping
- Projection model

AI

- Moneypal Genie architecture
 - Query model
 - RAG design
 - Sample prompts
-

Success Criteria

At the end of Week 1:

- Every business concept has a defined NAB representation.
- Every legacy data element has a documented path into the canonical model.
- Every source system can export data using a published CSV contract.
- Every DNBS reporting field is traceable back to one or more NAB events.
- Moneypal Genie has a clear architecture for answering natural language questions over NAB data with traceable, explainable results.
- The four workstreams integrate into a single coherent Genesis Layer architecture, providing a solid foundation for implementation in subsequent sprints.